
Del diseño lógico al diseño físico

Implementación de una base de datos - DDL

BASES DE DATOS I - FaCENA | UNNE

Implementación del diseño lógico

En la etapa final de **diseño lógico**, el objetivo principal es transformar el esquema conceptual de alto nivel (el diagrama Entidad-Relación) en el modelo de datos específico que soporta el SGBD elegido, el cual habitualmente es el **modelo relacional**.



SQL en el diseño lógico



El lenguaje SQL (Structured Query Language), y concretamente su subconjunto **DDL (Data Definition Language)**, es la herramienta formal mediante la cual se **traduce, implementa y formaliza las tablas, columnas, dominios y restricciones de integridad** directamente dentro del sistema gestor de bases de datos.

- **SQL** (Structured Query Language o Lenguaje de Consulta Estructurado) es un lenguaje de programación que permite realizar diversas operaciones sobre bases de datos relacionales.
- En la actualidad, el **estándar SQL ANSI** es considerado el estándar de facto de la inmensa mayoría de los sistemas de gestión de bases de datos (SGBD) comerciales.

SQL



El lenguaje **SQL puro (ANSI)** se clasifica en tres tipos principales de sentencias según la función que realizan.

- **DDL** (Data Definition Language / Definición de datos): Se utiliza para definir o modificar las estructuras de los objetos en la base de datos. Sus principales sentencias son: **CREATE, ALTER, DROP**.
- **DML** (Data Manipulation Language / Manipulación de datos): Se utiliza para trabajar directamente con los datos almacenados en las tablas. Sus sentencias principales incluyen: **SELECT, INSERT, UPDATE, DELETE**, Manipulación de transacciones (BEGIN TRANSAC, COMMIT, ROLLBACK).
- **DCL** (Data Control Language / Control de datos): Se encarga de administrar los permisos y privilegios de acceso a la información. Sus sentencias son: **GRANT, REVOKE**

DDL permite traducir e implementar formalmente el modelo de datos en el gestor a través de:

1. Definición de **Tablas, Columnas y Dominios**
2. Formalización de Restricciones de Integridad
3. Modificación y Gestión de objetos



Sentencias DDL

Lenguaje de Definición de Datos (Data Definition Language)

IMPLEMENTACIÓN FORMAL DEL MODELO DE DATOS: SQL DDL

El lenguaje DDL (Data Definition Language) es el subconjunto de SQL especializado en definir, modificar y eliminar las estructuras de una base de datos. Su evolución ha formalizado la integridad y portabilidad del esquema lógico a través de estándares ANSI/ISO.

DEFINICIÓN DE ESTRUCTURAS Y DOMINIOS

DECLARACIÓN DE TABLAS Y ESQUEMAS

El comando **CREATE TABLE** define la estructura base, permitiendo la creación atómica mediante **CREATE SCHEMA**



ESPECIFICACIÓN DE COLUMNAS Y TIPOS DE DATOS

123.37
947.92
0.00025

Tipos Numéricos
(INT, DECIMAL)

CHAR,
VARCHAR

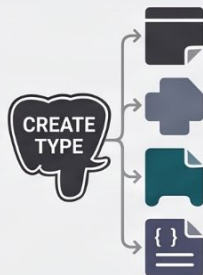
De Caracteres
(CHAR, VARCHAR)



Temporales
(DATE, TIMESTAMP)

DEFINICIÓN DE DOMINIOS DE USUARIO (UDT)

Permite unificar el diseño mediante **CREATE TYPE**, definiendo tipos de datos personalizados y reutilizables



COMPARATIVA DE TIPOS DE DATOS ENTEROS ESTÁNDAR

tinyint		1 byte
int		4 bytes
bigint		8 bytes

RESTRICCIONES DE INTEGRIDAD Y MODIFICACIÓN DE OBJETOS

FORMALIZACIÓN DE RESTRICCIONES DE INTEGRIDAD

Implementación de **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL** y **CHECK** para garantizar consistencia semántica

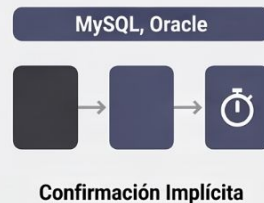
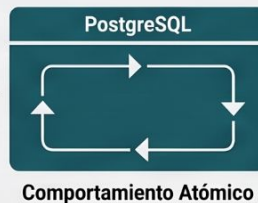


MODIFICACIONES SOBRE OBJETOS EXISTENTES

Uso de **ALTER TABLE** para cambios estructurales y **DROP TABLE** para la eliminación definitiva

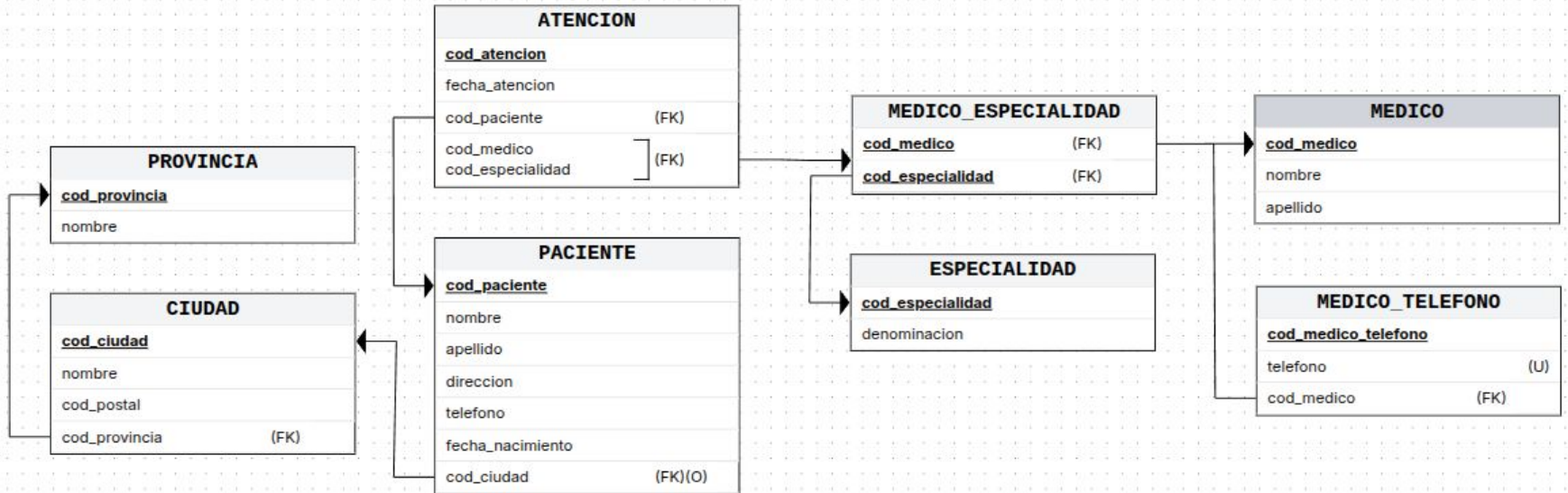


PARADIGMA DEL DDL TRANSACCIONAL



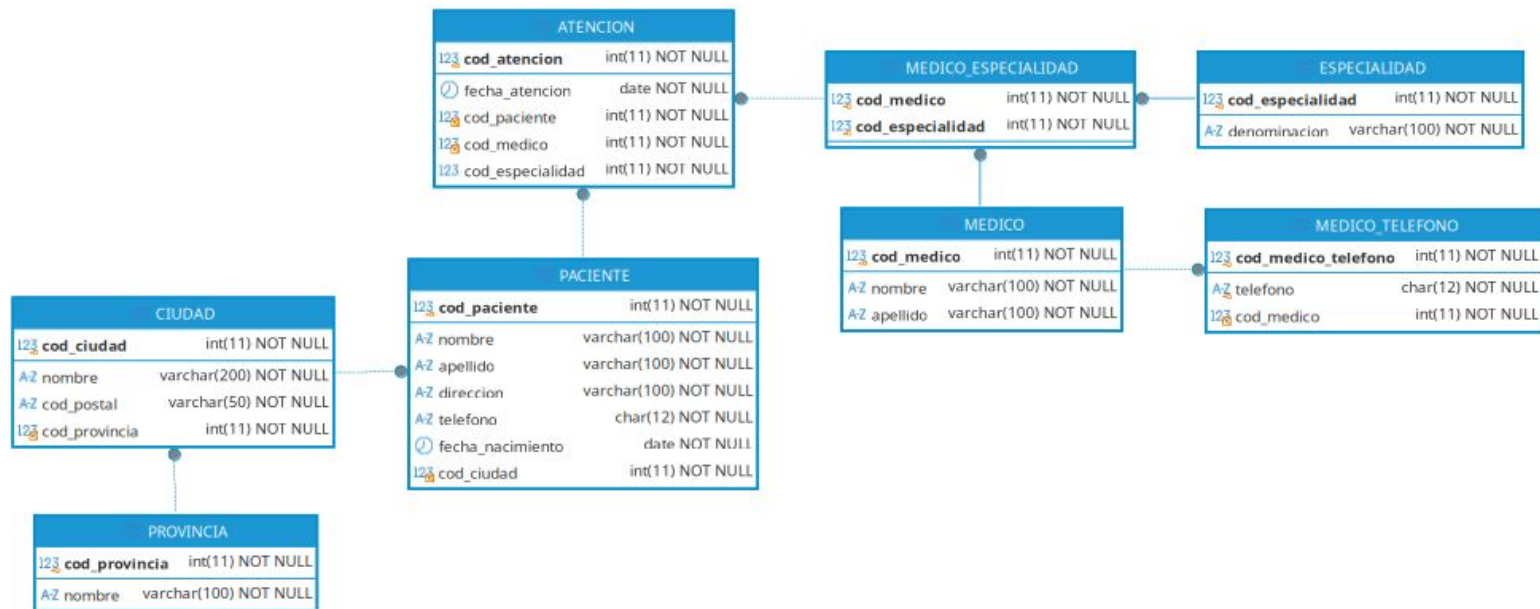
CASO DE ESTUDIO

Implementar el ejercicio 6 de la SERIE 1 - Clínica



CASO DE ESTUDIO

Resultado esperado de la implementación



SQL - CREATE DATABASE



La sentencia **CREATE** es el principal comando de SQL para definir datos. Se emplea para crear la propia base de datos o esquema (schema, en inglés), tablas, dominios, vistas, procedimientos y triggers.

```
-- Crear una base de datos  
CREATE DATABASE clinica;
```

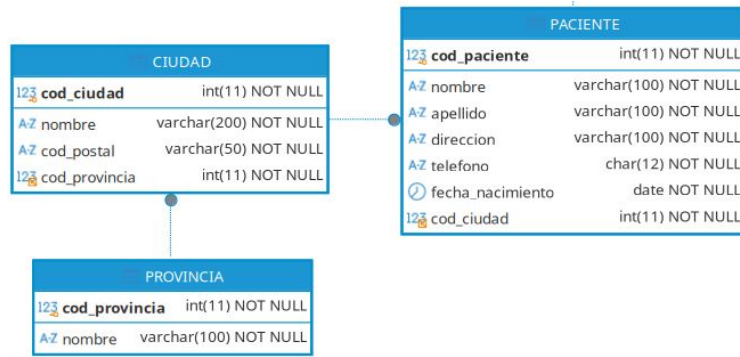
SQL - CREATE TABLE

Mediante el comando **CREATE TABLE** se crea una nueva **tabla** o relación, y se le **asigna un nombre y atributos**, así como las **restricciones iniciales**.

ORDEN DE CREACIÓN de las TABLAS: si las restricciones de clave foránea (FK) se incluyen en la sentencia **CREATE TABLE**, entonces es necesario considerar el orden de creación de las tablas en función de la dependencia parcial o total con las otras tablas.

```
CREATE TABLE PROVINCIA
(
  cod_provincia INT NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  CONSTRAINT PK_PROVINCIA PRIMARY KEY (cod_provincia)
);

CREATE TABLE CIUDAD
(
  cod_ciudad INT NOT NULL,
  nombre VARCHAR(200) NOT NULL,
  cod_postal VARCHAR(50) NOT NULL,
  cod_provincia INT NOT NULL,
  CONSTRAINT PK_CIUDAD PRIMARY KEY (cod_ciudad),
  CONSTRAINT FK_CIUDAD_PROVINCIA FOREIGN KEY
(cod_provincia) REFERENCES PROVINCIA(cod_provincia)
);
```



```
CREATE TABLE PACIENTE
(
  cod_paciente INT NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  direccion VARCHAR(100) NOT NULL,
  telefono CHAR(12) NULL,
  fecha_nacimiento DATE NOT NULL,
  cod_ciudad INT NULL,
  alta CHAR(1) NOT NULL DEFAULT 1,
  CONSTRAINT PK_PACIENTE PRIMARY KEY (cod_paciente),
  CONSTRAINT FK_PACIENTE_CIUDAD FOREIGN KEY (cod_ciudad)
REFERENCES CIUDAD(cod_ciudad)
);
```

SQL - Definición de Dominios y Tipos de Datos

Al declarar cada columna dentro de la sentencia `CREATE TABLE`, es **obligatorio** asignarle un **dominio** o **tipo de dato**. El dominio de una columna especifica el conjunto de valores válidos que se pueden almacenar en ella, acotando la naturaleza, el tipo y el tamaño de los datos (**escala, precisión y límites de los valores permitido**)

Categoría de Dominio	ANSI SQL (Estándar)	SQL Server (T-SQL)
Numéricos Exactos (Enteros)	INTEGER, SMALLINT, BIGINT	bigint (8 bytes), int (4 bytes), smallint (2 bytes), tinyint (1 byte)
Numéricos de Precisión Fija	NUMERIC(p,s), DECIMAL(p,s)	decimal(p,s), numeric(p,s) (precisión de 1 a 38 dígitos)
Numéricos Inexactos (Flotantes)	FLOAT, REAL, DOUBLE PRECISION	float, real
Cadenas de Caracteres (No-Unicode)	CHAR(n), VARCHAR(n)[5]	char(n) (longitud fija), varchar(n) (longitud variable)[6][7]
Cadenas de Caracteres (Unicode)	NCHAR(n), NVARCHAR(n)[5]	nchar(n), nvarchar(n) (utilizan el doble de almacenamiento en bytes)[7]
Fecha, Hora e Intervalos	DATE, TIME, TIMESTAMP, INTERVAL	datetime, smalldatetime[8]
Datos Semiestructurados y Avanzados	XML, JSON, LOB/CLOB, Arreglos (MDarray)	XML nativo (con validación XSD), Tipos Definidos por Usuario (CREATE TYPE), Tipos .NET CLR

SQL - ALTER TABLE



La sentencia **ALTER TABLE** modifica una definición de tabla, agrega o quita columnas y restricciones.

```
-- Agregar nueva columna
ALTER TABLE MEDICO ADD numero_matricula VARCHAR(50) NULL;
ALTER TABLE MEDICO ADD segundo_nombre VARCHAR(50) NULL;
ALTER TABLE MEDICO ADD estado CHAR(1) NULL;
```

```
-- Quitar una columna
ALTER TABLE MEDICO DROP COLUMN segundo_nombre;
```

```
-- Cambiar el tipo de datos
-- la sintaxis depende el SGBD
ALTER TABLE MEDICO ALTER COLUMN estado INT NULL;
```

SQL - NULL / NOT NULL



NULL se utiliza para representar que el valor de un atributo es **desconocido** o no está presente. No hay dos valores **NULL** que sean iguales entre sí, esto afecta a las operaciones DML que se realicen sobre los datos.

Durante la declaración de la estructura de los objetos en la base de datos (mediante instrucciones como **CREATE TABLE** o **ALTER TABLE**), definir la permisividad de nulos funciona como una **regla de dominio e integridad** para acotar qué datos son aceptables en cada columna.

Restricción NOT NULL : Especifica formalmente que una columna no puede quedar vacía ni recibir valores desconocidos, exigiendo que siempre se proporcione un dato válido

Clave Primaria (PRIMARY KEY): Requiere de forma obligatoria e implícita que todas las columnas que la formen sean **NOT NULL**.

SQL - Restricciones



Las **restricciones de integridad** (*integrity constraints*) en bases de datos relacionales son reglas lógicas y declarativas definidas mediante el sublenguaje DDL (*Data Definition Language*) para asegurar que los datos almacenados sean **válidos, exactos y consistentes**.

Restricciones de Clave

- **PRIMARY KEY (Clave Primaria - PK):** Define la columna o conjunto de columnas que identifican de forma única e inequívoca a cada fila de una tabla
- **FOREIGN KEY (Clave Foránea - FK):** Garantiza la **integridad referencial** estableciendo un vínculo entre una columna de una tabla hija y la clave primaria/única de una tabla padre
- **UNIQUE (Unicidad - UQ):** Asegura que no existan valores duplicados dentro de una columna o combinación de columnas

Restricciones de Dominio

- **NOT NULL / NULL**
- **DEFAULT (Valor por Defecto):** Asigna de forma automática un valor predeterminado a una columna cuando una instrucción de inserción (**INSERT**) omite explícitamente dicho campo
- **CHECK (Verificación):** Evalúa una expresión booleana o condición lógica sobre el valor de una o más columnas

SQL - ALTER TABLE

La sentencia **ALTER TABLE** permite agregar y borrar **RESTRICCIONES** en cada tabla

```
-- Agregar una restricción de unicidad
ALTER TABLE MEDICO
    ADD CONSTRAINT UQ_numero_matricula UNIQUE (numero_matricula);
```

```
-- Agregar una restricción no comprobada
ALTER TABLE MEDICO WITH NOCHECK
    ADD CONSTRAINT CK_MEDICO_estado CHECK (estado IN (0, 1))
```

```
-- Agregar una restricción de valor por defecto en una columna
-- es una de las instrucciones menos estandarizadas en SQL, depende el SGBD
ALTER TABLE MEDICO
    ADD CONSTRAINT DF_MEDICO_estado DEFAULT 1 FOR estado;
```

```
-- Borrar una restricción
ALTER TABLE MEDICO DROP CONSTRAINT CK_MEDICO_estado;
```

SQL - ALTER TABLE



Desactivar y activar restricciones en una tabla

```
-- Desactivar restricciones en toda la tabla
ALTER TABLE MEDICO NOCHECK CONSTRAINT ALL;
-- para una sola restricción
ALTER TABLE MEDICO NOCHECK CONSTRAINT CK_MEDICO_estado;
```

```
-- Activar restricciones nuevamente
ALTER TABLE MEDICO CHECK CONSTRAINT ALL;
ALTER TABLE MEDICO CHECK CONSTRAINT CK_MEDICO_estado;
```

SQL - ALTER TABLE

Desactivar y activar restricciones en una tabla

Tipo de Restricción	¿Se deshabilita con NOCHECK CONSTRAINT ALL?	Razón
CHECK	SI	Se desactivan las validaciones de valores en columnas.
FOREIGN KEY	SI	Se desactivan las validaciones de integridad referencial.
PRIMARY KEY	NO	Es la identidad primaria de la tabla y mantiene los índices únicos subyacentes.
UNIQUE	NO	Garantiza la unicidad de datos mediante un índice único que no se puede omitir temporalmente.
DEFAULT	NO	Los valores por defecto siguen aplicándose automáticamente al insertar filas sin ese valor.
NOT NULL	NO	Es una propiedad directa del tipo de dato de la columna, no una restricción deshabilitable.

SQL - DROP TABLE

Elimina la definición de una tabla y todos los datos, índices, restricciones y especificaciones de permisos asociados. **Realiza un borrado físico de la tabla.**

```
-- Eliminar tabla  
DROP TABLE MEDICO;
```

Para **borrar una tabla de la base de datos**, se debe tener en cuenta las dependencias de las tablas, es decir, las **restricciones de clave foránea (FK)** establecidas. Por lo cual, en este escenario, hay que considerar **el orden de borrado** de la estructura.

SQL - TRUNCATE TABLE



Restablece la estructura de la tabla a su estado original de creación. **Borra todos los datos** de la tabla.

```
-- Reinicia la tabla a su estado original, y borra todos los datos  
TRUNCATE TABLE MEDICO;
```

No se puede ejecutar **TRUNCATE** en una tabla que esté siendo referenciada por una **Foreign Key**, incluso si la tabla secundaria está completamente vacía (el motor no puede validar si al borrar los registros dejará datos huérfanos en otras tablas)

Analizar requerimiento - Caso CLÍNICA

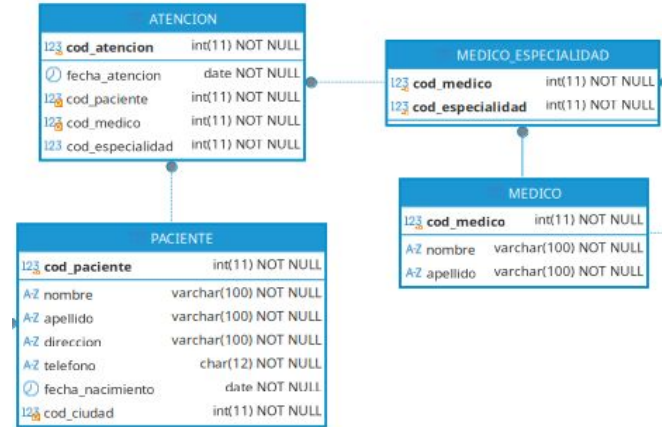
¿Cómo se trasladan los requerimientos en el diseño de la base de datos relacional?

RN099: En la clínica una persona puede estar registrada como médico y como paciente.

RN100: Un médico no puede ser atendido por sí mismo.

RN099: No es necesario controlar nada, debido a que se aceptan datos en las dos tablas, indistintamente

RN100: En el diseño lógico inicial del modelo relacional esta restricción no se puede reflejar. Es una restricción de implementación de la base de datos.



```
-- RN100. Suponemos que cod_paciente y cod_medico resguardan el DNI de la persona
ALTER TABLE ATENCION
    ADD CONSTRAINT CK_ATENCION_control CHECK (cod_paciente <> cod_medico)
```



Bibliografía

- Documentación del Aula Virtual